

PROJECT NOVA — Chapter 6 Design Brief

(Hand this to Claude Code, section by section)

HOW TO USE THIS DOCUMENT

Don't paste the whole thing at once. Work through ONE section at a time with Claude Code: paste a section, let it build, test in the browser, give feedback, then move to the next. Building in small steps is how you avoid breakage.

Before each big change, commit your work:

```
git add .
```

```
git commit -m "describe what you just finished"
```

CONTEXT (paste this first, once)

I'm building Project NOVA, an educational STEM game for Singapore upper-secondary students (MOE syllabus). It's mobile-first but must work on desktop. Tech: Vite + Phaser.js (vanilla JS) for frontend, Python FastAPI backend. Game code is in frontend/src/: main.js, scenes.js, effects.js.

Chapter 6 ("Signal") is the prototype chapter. The player calculates how to fire a signal beam to reach other survivor groups. It currently has:

- A title screen, a story briefing scene, and a gameplay scene
- Q1: angle of elevation (trigonometry), Q2: signal path length (Pythagoras)
- A hint system, on-screen number pad, correct/incorrect feedback
- A dark futuristic Command & Conquer visual style

Please read main.js, scenes.js and effects.js fully before making any changes, and build on the existing code rather than rewriting from scratch.

DESIGN PRINCIPLES (the "why" — keep these in mind throughout)

1. CONSEQUENCE OVER DECORATION. Visual impact comes from showing the RESULT of the student's decision (beam hits vs. overshoots), not from pretty pictures.
2. LET THEM MANIPULATE, NOT JUST ANSWER. The student should be able to drag/adjust and see live feedback, experimenting toward the answer — not compute blind.

3. RESPECT THE PLAYER. Fast to get into, no long tutorials, no patronising tone. Failure shows a result, never a lecture.
 4. MAKE IT THEIRS. Small customisation (name your settlement/crew, pick a colour) creates ownership and return visits.
 5. SOCIAL COMPARISON. Knowing where you rank ("top 5 in class", "Wei Jie beat your score") is a huge motivator — even without live multiplayer.
 6. COMPETITION MUST REWARD LEARNING, NOT SABOTAGE OR SPEED-MASHING. Any "attack" on a friend must itself require solving a harder problem.
 7. SHORT SESSIONS, CLEAR PROGRESS. A chapter should be finishable in one sitting with a visible sense of moving forward.
-

PRIORITY 1 — THE CORE INTERACTION (build this first)

This is the single highest-impact change. Turn the static "calculate and submit" into a live, manipulable mechanic:

- Add a draggable angle dial/slider that sets the beam's firing angle.
- As the student drags, a beam line on screen updates in REAL TIME to show where it would fire.
- A "FIRE" button commits the shot.
- On fire: animate the beam arcing out. If the angle is correct (within tolerance), it bounces off the ionosphere and reaches the settlement — show lights coming on. If wrong, it overshoots into space and dissipates — show the settlement stays dark.
- Keep the number-pad input as an alternative for students who want to calculate exactly, but the drag-to-aim is the primary, satisfying interaction.

Make the FIRE moment feel punchy: a brief charge-up, a clear fire, a visible hit-or-miss result.

PRIORITY 2 — TRIGONOMETRY CONTENT (Chapter 6 scope)

Chapter 6 should cover, scaled by difficulty:

- Right-angled trig ($\tan/\sin/\cos$)
- Angle of elevation AND depression (e.g. tower looking down into a valley = depression)
- Bearings (direction to another settlement, e.g. "on a bearing of 120 degrees")

Do NOT put sine rule or cosine rule here — those need non-right triangles and will be saved for a later "triangulation of multiple settlements" chapter.

All of the above are MOE E-Math topics, so they work for students who don't take Physics. IMPORTANT: if unsure whether a topic belongs to E-Math or A-Math, flag it rather than guessing.

Difficulty tiers (for adaptive difficulty later):

- Entry: given two sides, find the angle (one step)
 - Mid: multi-step (find a missing side first, then the angle)
 - Higher: bearings combined with trig; student builds the triangle from a word problem
 - Stretch: student must DECIDE which method applies, not be told
-

PRIORITY 3 — EDUCATIONAL LAYER

- After a CORRECT answer, show the worked solution (the method), not just "correct" — this reinforces it for studying.
 - Randomise the question values (e.g. distance and ionosphere height) each attempt, keeping the math valid, so students can't just memorise one answer.
 - Track per attempt: accuracy, number of attempts, whether a hint was used.
 - Small XP penalty for using a hint.
 - Map each question to its MOE syllabus topic (store this so it can be shown to schools/parents later).
-

PRIORITY 4 — FEEL / POLISH

- Sound: typing SFX in briefing, ambient music, button clicks, success/fail stings.
 - A satisfying signal-firing animation (the beam payoff described in Priority 1).
 - Smooth transitions between scenes.
 - Light customisation: let the player name their settlement/crew.
-

PRIORITY 5 — SOCIAL LAYER (build LAST, it's the most complex)

Build in this order, only after single-player is fun:

1. Asynchronous leaderboard: solve a mission, score (accuracy + speed + hints used) posts to a board; friends try to beat it on the same scenario.
2. Class ranking display ("you're top 5 in your class").

3. ONLY THEN consider a live "attack" system where attacking a friend requires solving a harder problem (and defending requires solving one too). The currency is correct reasoning, never button-mashing or pure sabotage.

This layer needs accounts, a database, and real-time sync — it will take far longer than everything else. Do not start it until the single-player loop is proven fun.

ART ASSETS (note for when you're ready)

As a solo dev: use AI-generated background images (the wasteland, the tower, the ionosphere) plus code-drawn effects (beam, glow, particles). One strong background per chapter + code effects gets most of the impact at near-zero cost. Don't block progress waiting for art — greybox it first, add art later.

SUGGESTED BUILD ORDER (summary)

1. Core draggable-beam interaction (Priority 1)
2. Trig content + difficulty (Priority 2)
3. Educational layer — worked solutions, randomised values, tracking (Priority 3)
4. Feel/polish — sound, animation, customisation (Priority 4)
5. Social layer — leaderboard first, attack system much later (Priority 5)

Finish and test each before starting the next.